L16 - 完整模型组装

> *"乐高积木拼出飞机"*

📌 本节目标

1. 回顾所有组件并理解它们的功能
2. 理解 Transformer Block 的组装方式
3. 掌握 MiniMindForCausalLM 的完整架构
4. **学会计算模型参数量**（面试必考）
5. 理解自回归生成（generate）的过程

📚 前置知识

- Phase 2 所有组件课程（Embedding、Attention、FFN、RMSNorm 等）
- Phase 3 训练课程（L11-L15）
- 基本的矩阵运算

---

1. 组件回顾

在开始组装之前，让我们快速回顾每个组件的功能：

1.1 组件清单

组件	功能	输入 → 输出
Embedding	将 token ID 映射为向量	[batch, seq_len] → [batch, seq_len, dim]
RMSNorm	归一化，稳定训练	[batch, seq_len, dim] → [batch, seq_len, dim]
Attention (GQA)	捕获 token 间的关系	[batch, seq_len, dim] → [batch, seq_len, dim]

组件	功能	输入 → 输出
FFN (SwiGLU)	非线性变换，存储知识	[batch, seq_len, dim] → [batch, seq_len, dim]
残差连接	缓解梯度消失	x + sublayer(x)
lm_head	将向量映射回词表空间	[batch, seq_len, dim] → [batch, seq_len, vocab_size]

1.2 MiniMind 的核心超参数

超参数	值	含义
`dim`	768	隐藏层维度
`n_layers`	8	Transformer 层数
`n_heads`	16	Query 注意力头数
`n_kv_heads`	8	KV 注意力头数（GQA）
`vocab_size`	6400	词表大小
`max_seq_len`	512	最大序列长度
`intermediate_size`	2048	FFN 中间维度

---

2. Transformer Block 的组装

2.1 一个 Block 的结构

一个 Transformer Block 包含以下组件：

```
输入 x
  │
  ├──→ RMSNorm ──→ Attention ──→ +  ← 残差连接
  │                              │
  │←─────────────────────────────┘
  │
  ├──→ RMSNorm ──→ FFN ──→ +  ← 残差连接
  │                        │
  │←───────────────────────┘
  │
输出
```

2.2 为什么是 Pre-Norm?

MiniMind 使用 **Pre-Norm**（归一化在子层之前），而不是原始 Transformer 的 Post-Norm：

```
# Pre-Norm（MiniMind 使用）
x = x + Attention(RMSNorm(x))
x = x + FFN(RMSNorm(x))

# Post-Norm（原始 Transformer）
x = RMSNorm(x + Attention(x))
x = RMSNorm(x + FFN(x))
```

Pre-Norm 的优势： - 训练更稳定，不容易出现梯度消失/爆炸 - 不需要复杂的学习率 warmup 策略 - 几乎所有现代 LLM 都使用 Pre-Norm

2.3 代码实现

```
class TransformerBlock(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.attention = Attention(config)
        self.feed_forward = FeedForward(config)
        self.attention_norm = RMSNorm(config.dim)
        self.ffn_norm = RMSNorm(config.dim)

    def forward(self, x, freqs_cis, mask=None):
        # Attention + 残差
        h = x + self.attention(self.attention_norm(x), freqs_cis, mask)
        # FFN + 残差
        out = h + self.feed_forward(self.ffn_norm(h))
        return out
```

---

3. 完整模型架构

3.1 MiniMindConfig

配置类定义了模型的所有超参数：

```
class MiniMindConfig:
    dim: int = 768
    n_layers: int = 8
    n_heads: int = 16
    n_kv_heads: int = 8
    vocab_size: int = 6400
    max_seq_len: int = 512
```

```
    intermediate_size: int = 2048
    norm_eps: float = 1e-5
```

3.2 MiniMindModel（骨干网络）

```
class MiniMindModel(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.embed_tokens = nn.Embedding(config.vocab_size, config.dim)
        self.layers = nn.ModuleList([
            TransformerBlock(config) for _ in range(config.n_layers)
        ])
        self.norm = RMSNorm(config.dim)
```

3.3 MiniMindForCausalLM（完整模型）

```
class MiniMindForCausalLM(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.model = MiniMindModel(config)
        self.lm_head = nn.Linear(config.dim, config.vocab_size, bias=False)

        # 权重共享：lm_head 和 embed_tokens 共享权重
        self.lm_head.weight = self.model.embed_tokens.weight

    def forward(self, input_ids):
        h = self.model(input_ids)    # [batch, seq, dim]
        logits = self.lm_head(h)      # [batch, seq, vocab_size]
        return logits
```

3.4 权重共享（Weight Tying）

MiniMind 的 Embedding 层和 lm_head 层**共享权重**。即 `lm_head.weight = embed_tokens.weight`。

为什么要共享？ 1. **减少参数量**：省去一个 vocab_size × dim 的矩阵（约 5M 参数） 2. **语义一致性**：输入和输出使用相同的词向量空间 3. **提升效果**：实验表明权重共享通常能提升小模型的效果

注意：权重共享后，Embedding 的参数和 lm_head 的参数是同一个张量。反向传播时，来自两端的梯度会**累加**到这个共享权重上。

---

4. 完整数据流

4.1 数据流图

```
input_ids: [batch_size, seq_len]
    │
    ▼
┌──────────────┐
│ embed_tokens │  vocab_size × dim = 6400 × 768
└──────────────┘
       │
       │  [batch, seq, 768]
       ▼
┌──────────────────────┐
│  Transformer Block 1 │
│  ├─ RMSNorm          │
│  ├─ Attention (GQA)  │
│  ├─ Residual         │
│  ├─ RMSNorm          │
│  ├─ FFN (SwiGLU)     │
│  └─ Residual         │
└──────────────────────┘
           │  [batch, seq, 768]
           ▼
       ... × 8 层 ...
           │
           ▼
┌──────────────────────┐
│  Transformer Block 8 │
└──────────────────────┘
           │  [batch, seq, 768]
           ▼
┌────────────┐
│  RMSNorm   │  最终归一化
└────────────┘
      │  [batch, seq, 768]
      ▼
┌──────────┐
│  lm_head │  dim × vocab_size = 768 × 6400
└──────────┘
     │  [batch, seq, 6400]
     ▼
  logits → Softmax → 概率分布
```

4.2 维度变化追踪

以 batch_size=2, seq_len=10 为例，追踪数据的维度变化：

```
输入 input_ids:                [2, 10]          (token IDs)
经过 embed_tokens:             [2, 10, 768]      (词向量)
经过 Block 1 的 RMSNorm:      [2, 10, 768]      (归一化)
经过 Block 1 的 Attention:     [2, 10, 768]      (注意力输出)
经过 Block 1 的残差连接:       [2, 10, 768]      (+ 原始输入)
经过 Block 1 的 RMSNorm:      [2, 10, 768]      (归一化)
经过 Block 1 的 FFN:           [2, 10, 768]      (FFN输出)
```

```
经过 Block 1 的残差连接:       [2, 10, 768]       (+ 注意力输出)
...经过 8 个 Block...
经过最终 RMSNorm:              [2, 10, 768]       (最终归一化)
经过 lm_head:                  [2, 10, 6400]      (logits)
```

注意：在所有 Transformer Block 中，数据的形状始终是 `[batch, seq, 768]`，**维度不变**。这是 Transformer 架构的一个优美特性。

---

5. 参数量计算

5.1 逐组件拆解

这是**面试必考题**。让我们精确计算 MiniMind 的每个组件的参数量。

约定：dim=768, n_heads=16, n_kv_heads=8, vocab_size=6400, intermediate_size=2048, n_layers=8

Embedding 层

```
embed_tokens: vocab_size × dim = 6400 × 768 = 4,915,200
```

每层 Attention

```
head_dim = dim / n_heads = 768 / 16 = 48

Q 投影: dim × (n_heads × head_dim) = 768 × 768 = 589,824
K 投影: dim × (n_kv_heads × head_dim) = 768 × (8×48) = 768 × 384 = 294,912
V 投影: dim × (n_kv_heads × head_dim) = 768 × 384 = 294,912
O 投影: (n_heads × head_dim) × dim = 768 × 768 = 589,824

Attention 总计: 589,824 + 294,912 + 294,912 + 589,824 = 1,769,472
```

每层 FFN（SwiGLU）

SwiGLU 有三个投影矩阵：gate、up、down

```
gate: dim × intermediate_size = 768 × 2048 = 1,572,864
up:   dim × intermediate_size = 768 × 2048 = 1,572,864
down: intermediate_size × dim = 2048 × 768 = 1,572,864

FFN 总计: 1,572,864 × 3 = 4,718,592
```

每层 RMSNorm

每个 Block 有 2 个 RMSNorm：

```
每个 RMSNorm: dim = 768 个参数（可学习的缩放因子 γ）
每层 RMSNorm: 768 × 2 = 1,536
```

最终 RMSNorm

```
最终 RMSNorm: 768
```

lm_head

```
lm_head: dim × vocab_size = 768 × 6400 = 4,915,200
但与 embed_tokens 共享权重，所以不额外计算！
```

5.2 总参数量

```
Embedding:          4,915,200
8 层 Attention:     1,769,472 × 8 = 14,155,776
8 层 FFN:           4,718,592 × 8 = 37,748,736
8 层 RMSNorm:       1,536 × 8 = 12,288
最终 RMSNorm:       768
lm_head:            0（共享权重）
──────────────────────────────────
总计:               56,832,768 ≈ 56.8M
```

加上 RoPE 的频率缓存等非训练参数，MiniMind 声称的 **64M** 参数（不同配置略有差异）。

5.3 参数分布分析

```
Attention: 14.16M (24.9%)  ← 约 1/4
FFN:       37.75M (66.4%)  ← 约 2/3 ← 参数量大户！
Embedding: 4.92M  (8.7%)
其他:      0.01M  (0.0%)
```

FFN 占了模型参数的约 2/3。这也是为什么有些轻量级优化主要针对 FFN（如 MoE 混合专家）。

---

6. 自回归生成（Generate）

6.1 基本原理

自回归生成就是**逐个 token 生成**：

```
输入: "什么是"
步骤1: 模型("什么是") → 预测 "机"
步骤2: 模型("什么是机") → 预测 "器"
步骤3: 模型("什么是机器") → 预测 "学"
步骤4: 模型("什么是机器学") → 预测 "习"
步骤5: 模型("什么是机器学习") → 预测 "？"
...
直到生成 <|im_end|> 或达到最大长度
```

每一步，模型输出 vocab_size 维的 logits，通过某种策略选出下一个 token。

6.2 解码策略

贪心搜索（Greedy Search）

每步选概率最高的 token：

```
next_token = logits.argmax(dim=-1)
```

优点：简单、确定性。缺点：生成文本单调、缺乏多样性。

随机采样（Sampling）

按概率分布随机采样：

```
probs = F.softmax(logits / temperature, dim=-1)
next_token = torch.multinomial(probs, num_samples=1)
```

温度 temperature 控制多样性： - temperature < 1：分布更集中，生成更确定 - temperature > 1：分布更均匀，生成更多样 - temperature = 0：退化为贪心搜索

Top-k 采样

只从概率最高的 k 个 token 中采样：

```
topk_logits, topk_indices = logits.topk(k)
topk_probs = F.softmax(topk_logits / temperature, dim=-1)
next_token_idx = torch.multinomial(topk_probs, num_samples=1)
next_token = topk_indices.gather(-1, next_token_idx)
```

k 值控制候选范围：k=1 就是贪心，k=vocab_size 就是全采样。

Top-p 采样（Nucleus Sampling）

选择概率累积超过 p 的最小 token 集合：

```
sorted_logits, sorted_indices = logits.sort(descending=True)
sorted_probs = F.softmax(sorted_logits / temperature, dim=-1)
cumulative_probs = sorted_probs.cumsum(dim=-1)

# 找到累积概率超过 p 的位置
mask = cumulative_probs - sorted_probs > p
sorted_logits[mask] = -float('inf')

# 在剩余的 token 中采样
probs = F.softmax(sorted_logits / temperature, dim=-1)
next_token_idx = torch.multinomial(probs, num_samples=1)
next_token = sorted_indices.gather(-1, next_token_idx)
```

Top-p 的优势：自适应候选集大小。当模型很确定时，候选集小；当模型不确定时，候选集大。

6.3 实际使用中的组合

实际使用时通常组合多种策略：

```
# 典型配置
temperature = 0.7
top_k = 50
top_p = 0.9
repetition_penalty = 1.1
```

6.4 MiniMind 的 generate 方法

```
def generate(self, input_ids, max_new_tokens=100, temperature=0.7, top_p=0.9):
    """自回归生成"""
    for _ in range(max_new_tokens):
        # 前向传播，获取最后一个位置的 logits
        logits = self.forward(input_ids)[:, -1, :]  # [batch, vocab_size]

        # 温度缩放
        logits = logits / temperature
```

```
        # Top-p 采样
        next_token = top_p_sample(logits, p=top_p)

        # 拼接到输入序列
        input_ids = torch.cat([input_ids, next_token.unsqueeze(-1)], dim=-1)

        # 遇到结束符则停止
        if next_token.item() == eos_token_id:
            break

    return input_ids
```

---

7. MiniMind 源码解读

7.1 关键文件

- `model/model.py`：模型定义，包含 MiniMindModel、MiniMindForCausalLM
- `model/model_lora.py`：LoRA 版本的模型

7.2 MiniMindModel 类

这是模型的骨干网络，包含 Embedding、所有 Transformer Block 和最终的 RMSNorm：

```
class MiniMindModel(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.embed_tokens = nn.Embedding(config.vocab_size, config.dim)
        self.layers = nn.ModuleList([
            TransformerBlock(config) for _ in range(config.n_layers)
        ])
        self.norm = RMSNorm(config.dim, eps=config.norm_eps)
        # 预计算 RoPE 频率
        self.freqs_cis = precompute_freqs_cis(...)

    def forward(self, input_ids, start_pos=0):
        h = self.embed_tokens(input_ids)

        # 获取 RoPE 频率
        freqs_cis = self.freqs_cis[start_pos:start_pos + seq_len]

        # 通过所有 Transformer Block
        for layer in self.layers:
            h = layer(h, freqs_cis)

        # 最终归一化
        h = self.norm(h)
        return h
```

7.3 MiniMindForCausalLM 类

这是完整的因果语言模型，在骨干网络之上加了 lm_head：

```
class MiniMindForCausalLM(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.config = config
        self.model = MiniMindModel(config)
        self.lm_head = nn.Linear(config.dim, config.vocab_size, bias=False)

        # 权重共享
        self.lm_head.weight = self.model.embed_tokens.weight

    def forward(self, input_ids, labels=None):
        h = self.model(input_ids)
        logits = self.lm_head(h)

        loss = None
        if labels is not None:
            loss = F.cross_entropy(
                logits[..., :-1, :].reshape(-1, self.config.vocab_size),
                labels[..., 1:].reshape(-1),
                ignore_index=-100
            )
        return logits, loss
```

---

8. 从零到完成的训练全景

回顾整个 Phase 3，MiniMind 的完整训练流程如下：

```
第1步：准备数据
  pretrain_t2t_mini.jsonl → DataLoader → 预训练数据
  sft_t2t_mini.jsonl → DataLoader → SFT 数据

第2步：预训练（L12）
  随机初始化模型 → Next Token Prediction → 学会语言规律
  产出：pretrain_model.pt

第3步：监督微调 SFT（L13）
  加载预训练模型 → 多轮对话训练（带 Loss Mask） → 学会对话
  产出：sft_model.pt

第4步（可选）：LoRA 微调（L14）
  加载 SFT 模型 → 冻结原始权重 + LoRA 旁路 → 领域适配
  产出：lora_adapter.pt → 合并后 merged_model.pt

第5步（可选）：知识蒸馏（L15）
  大模型（教师） → 指导小模型（学生）训练
  产出：distilled_model.pt
```

```
第6步（可选）：DPO 偏好对齐
  加载 SFT 模型 → 对比学习（chosen vs rejected） → 偏好对齐
  产出：dpo_model.pt

最终：模型推理
  加载最终模型 → generate → 对话输出
```

🎤 面试考点

Q1: 如何计算模型参数量？（必考）

答：逐组件计算： 1. **Embedding**: vocab_size × dim 2. **每层 Attention**: (dim × dim) + 2 × (dim × kv_dim) + (dim × dim)，其中 kv_dim = n_kv_heads × head_dim 3. **每层 FFN (SwiGLU)**: 3 × (dim × intermediate_size) 4. **每层 RMSNorm**: 2 × dim 5. **最终 RMSNorm**: dim 6. **lm_head**: 如果与 Embedding 共享权重则不额外计算

总参数 = Embedding + n_layers × (Attention + FFN + RMSNorm) + 最终 RMSNorm。MiniMind 约 56.8M 参数。

Q2: 数据在模型中的完整流动路径是什么？

答： input_ids → Embedding（token ID 变向量）→ 8 层 Transformer Block（每层：RMSNorm → Attention → 残差 → RMSNorm → FFN → 残差）→ 最终 RMSNorm → lm_head（向量变 logits）→ Softmax → 概率分布。整个过程中隐藏维度始终为 768，只有最后 lm_head 将维度从 768 映射到 vocab_size=6400。

Q3: 自回归生成的过程？

答：自回归生成是逐个 token 生成的过程。每一步将当前所有 token 送入模型，取最后一个位置的 logits，通过解码策略（贪心、Top-k、Top-p）选出下一个 token，拼接到序列末尾。重复这个过程直到生成结束符或达到最大长度。每次只生成一个 token，因此生成速度与序列长度成正比。

Q4: 什么是权重共享？有什么好处？

答：权重共享是指 Embedding 层和 lm_head 层使用同一个权重矩阵。好处：(1) 减少参数量（对 MiniMind 节省约 5M 参数）；(2) 输入和输出在同一个语义空间中，语义一致性更好；(3) 实验表明对小模型效果有提升。

Q5: 为什么 FFN 占了模型参数的大部分?

答：FFN 有 3 个投影矩阵（SwiGLU 架构），每个是 dim × intermediate_size。而 intermediate_size 通常是 dim 的 2-4 倍（MiniMind 中是 2048 / 768 ≈ 2.67 倍）。因此每层 FFN 的参数量约为 3 × dim × intermediate_size，远大于 Attention 的参数量。FFN 被认为是模型存储"知识"的主要组件。

Q6: Top-k 和 Top-p 采样有什么区别?

答： - **Top-k**：固定候选集大小为 k。无论模型多确定或不确定，都只从前 k 个 token 中采样。 - **Top-p（Nucleus）**：自适应候选集大小。选择概率累积超过 p 的最小 token 集合。模型确定时候选少，不确定时候选多。 - Top-p 通常更灵活，是目前主流选择。实际使用中常组合使用。

✅ 自测题

1. 不看参考，手动计算 MiniMind 各组件的参数量。最终结果应约为 56-64M。
2. 画出完整的数据流图，标注每个位置的张量维度（假设 batch=1, seq_len=20）。
3. 实现一个简化版的 `generate` 函数，支持贪心搜索和 Top-p 采样。
4. 如果把 n_layers 从 8 增加到 16，参数量会变成多少?
5. 如果不使用权重共享，参数量会增加多少?
6. 回顾 L11-L16，用自己的话描述 MiniMind 从数据准备到模型推理的完整流程。

🎉 Phase 3 总结

恭喜你完成了 Phase 3：训练全流程！让我们回顾你掌握的技能：

课程	核心知识
L11 数据处理	数据格式、Loss Mask、梯度累积
L12 预训练	Next Token Prediction、交叉熵、学习率调度、AMP
L13 SFT	Chat Template、Loss Mask 实现、SFT vs 预训练
L14 LoRA	低秩分解、参数高效微调、权重合并
L15 知识蒸馏	黑盒/白盒蒸馏、温度、KL 散度

课程	核心知识
L16 模型组装	完整架构、参数量计算、自回归生成

你现在已经具备了从零理解一个 LLM 项目的全部知识。回到 MiniMind 的代码，你应该能读懂每一行。

> *"纸上得来终觉浅，绝知此事要躬行。"*

建议接下来：亲手跑一遍 MiniMind 的训练流程，从预训练到 SFT 到推理，把理论变成实践。